

1. Operating System (OS) kya hai aur iska Position

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference Note: Yeh topic aapke dwara provide ki gayi **Image 1** par aadharit hai jisme OS ki definition, concentric position diagram aur computer system ke 4 components explain kiye gaye hain.

1.1 Operating System ki Basic Definition

- **Master Program:** Operating System ek aisa **program / system software** hai jo computer ke physical hardware (H/W) ko poori tarah manage aur control karta hai.
- **Intermediary / Bridge Role:** Yeh user aur raw computer hardware ke beech ek **intermediary (madhyasth)** ka kaam karta hai. Agar OS na ho, toh user ko computer ke binary circuits, registers aur memory buses se direct baat karni padegi jo aam insaan ke bas ki baat nahi hai.
- **Basis for Application Programs:** OS ek aisa basic foundation provide karta hai jiske upar saare application software (jaise Web Browser, MS Office, Code Editors, Games) install hokar run kar sakte hain.
- **User-Friendly Environment:** OS ek convenient aur easy-to-use environment taiyar karta hai jisme user bina kisi technical complexity ke apne programs execute kar sake.

1.2 Diagram Reference & Handwritten Notes (Image 1)

Original Image 1 mein ek concentric circle (onion-like) diagram diya gaya hai jiska title hai Fig: - Position of Operating System. Image 1 ke right side mein 3 items/entities note kiye gaye hain:

Fig 1.1: Position of Operating System (Concentric View)

Reference: Image 1 [Position of OS & 3 Entities Architecture]

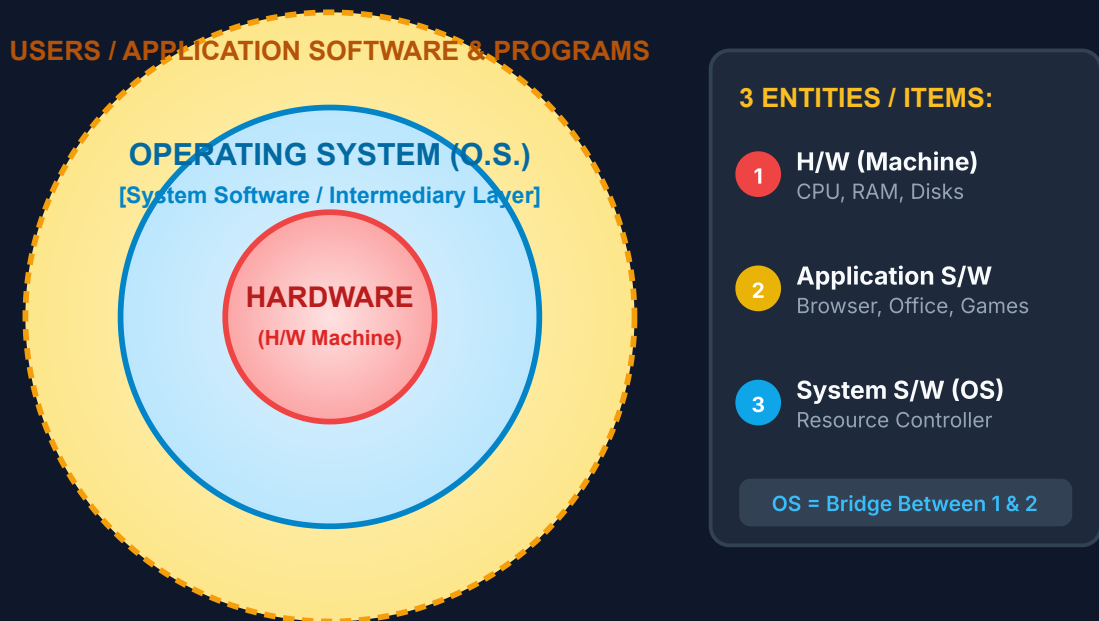


Fig 1.1: Position of Operating System & 3 Core Entities Architecture

1.3 Computer System ke 3 Main Entities (Items)

- **1.3.1 Hardware (H/W - Machine):** Physical components (CPU, RAM, Hard Disk, Motherboard, Graphics Card). Yeh computing capability aur raw processing power provide karta hai.
- **1.3.2 Application Software (Application S/W):** Specific end-user problems ko solve karne ke liye banaye gaye programs (e.g., Browser, Word Processor, Media Player, Games).
- **1.3.3 System Software (OS & Utilities):** Hardware aur application software ke beech coordination banata hai aur resource allocation control karta hai (~50% core management share).

1.4 Computer System ke 4 Core Components

Computer system ko 4 mukhya bhaagon mein divide kiya jata hai:

1. **The Hardware (H/W):** Basic computing resources provide karta hai (CPU, Memory, I/O Devices).
2. **The Operating System (O.S.):** Hardware ke use ko control aur coordinate karta hai different users aur application programs ke beech.
3. **The Application Programs:** Define karta hai ki hardware resources ka use karke user ki computing problems ko kaise solve kiya jaye.
4. **The Users:** End-users jo computer ko use karte hain (Humans, remote machines, ya automated background bots).

2. Computer System ka Abstract View aur Component Hierarchy

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concept: Jaise humne **Concept 1.4** mein dekha ki computer system ke 4 components hote hain (Hardware, OS, Application Programs, Users), yeh topic **Image 2** par aadharit hai jo in components ka layered **Abstract View** aur right side mein **Translators** ka flow explain karta hai.

2.1 Abstract View of Components (Layered Architecture)

- **Layer 1 (Bottom Most) - Computer Hardware (H/W):** Core machine jisme CPU, Main Memory (RAM), aur I/O devices aate hain.
- **Layer 2 - Operating System (OS):** Hardware ke theek upar rehta hai aur hardware ko wrap karke safe, uniform services deta hai.
- **Layer 3 - System & Application Programs:** OS ke upar different utilities run hoti hain: COMPILER, ASSEMBLER, TEXT EDITOR, DATABASES, SPREADSHEETS, GAMES.
- **Layer 4 (Top Most) - Multiple Users:** User 1, User 2, ... User N jo concurrently system use kar rahe hote hain.

2.2 Translators (Handwritten Note Reference from Image 2)

Image 2 ke top-right corner mein likha hai: TRANSLATORS :- 1) ASSEMBLER 2) COMPILER, INTERPRETER | HLL -> 0, 1.

- **Translators ki Zaroorat:** Computer Hardware (CPU) sirf aur sirf **Machine Language (0 aur 1 / Binary Bits)** samajhta hai. Hum insaan high-level English-like code likhte hain (C, C++, Java, Python). Translators ka kaam iss Human-readable code ko Computer-readable binary bits (0, 1) mein convert karna hai.
- **2.2.1 Assembler:** Assembly Language (Low-Level Language jaise MOV AX, BX) ko seedhe Machine Language (0 aur 1) mein convert karta hai.
- **2.2.2 Compiler:** High-Level Language (HLL jaise C/C++) ke poore code ko ek saath compile karke machine code generate karta hai.
- **2.2.3 Interpreter:** High-Level Language (jaise Python, JS) ko line-by-line interpret karke execute karta hai.

Fig 2.1: Abstract View of Computer System Components

Reference: Image 2 [Layered Components & Translators Architecture]

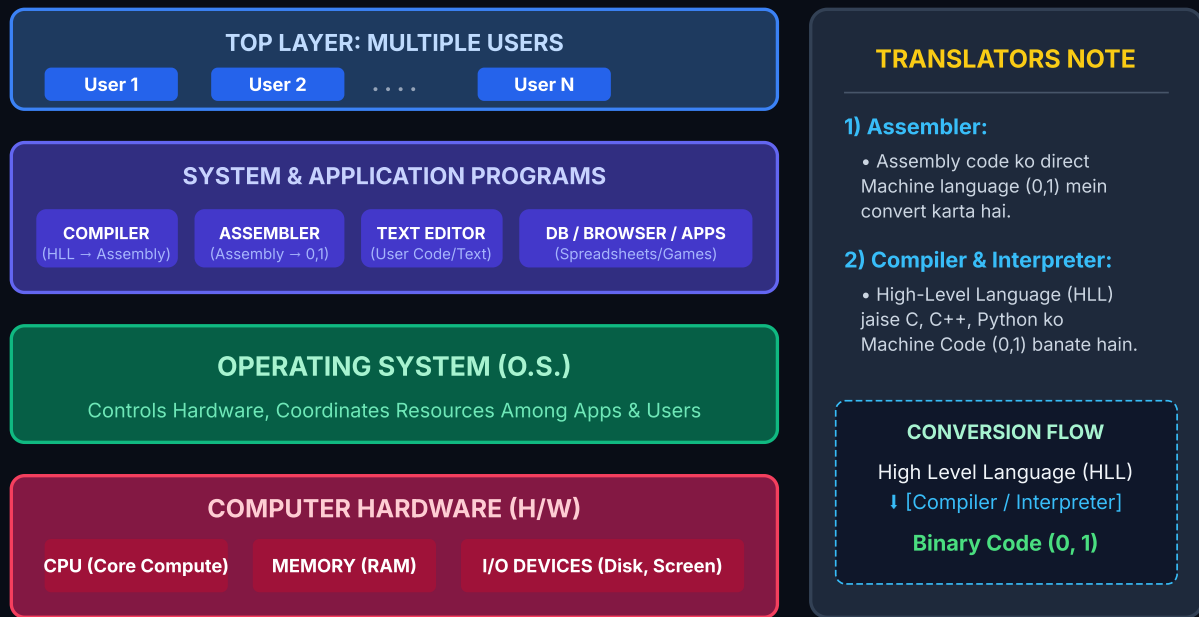


Fig 2.1: Abstract View of Computer System & Translators Flow

2.3 Hardware Resources (H/W Deep Dive)

- **(a) The Memory (RAM):** Programs ko execute hone ke liye memory chahiye hoti hai. Variables aur active data yahin store hota hai.
- **(b) Input & Output (I/O) Devices:** Disks (Storage), Keyboard, Mouse, Monitor, Network cards jinke zariye data in aur out hota hai.
- **(c) CPU (Central Processing Unit):** Basic computing resources aur calculations execute karne wala brain.

2.4 Application Programs vs OS Coordination

Application programs (Word, Spreadsheets, Games, DBMS) solve user computing problems. OS in programs ke beech hardware resources ka coordination karta hai taaki koi application hardware ko monopolize na kare aur system stable bana rahe.

3. User's aur System's Point of View of Operating System

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concepts: Concept 1.2 mein OS ko intermediary aur Concept 2.1 mein Abstract View dekha gaya tha. Yeh topic **Image 3** par aadharit hai jisme OS ko do mukhya perspectives se dekha jata hai: **User View** aur **System View (Resource Manager)**.

3.1 User View (End-User Perspective)

- **Primary Goal: Convenience, Usability & Responsiveness:** User ke liye OS ka matlab hai aasan aur bina rukawat computer chalana.
- **Interface Modes:** GUI (Windows, icons), CLI (Terminal commands), aur APIs (Developers ke liye).
- **User Focus:** Running applications, managing files, configuring settings, and interacting with peripherals.

3.2 System View (Resource Manager Perspective)

Image 3 handwritten note: "**Resource Manager**" (marked very important).

- **Primary Goal: Efficiency, Control & Resource Allocation:** System perspective se OS ek Resource Allocator hai. Hardware limited hai (CPU cores, RAM, I/O bandwidth). OS ka kaam hai fair aur collision-free distribution karna.
- **Core Components:** Kernel, Device Drivers, Memory Management, Process Management, File Systems, Networking Protocols.
- **Emphasis:** Efficiency, fair resource allocation, security aur stability.

Fig 3.1: Dual Perspectives of Operating System

Reference: Image 3 [User View vs System View / Resource Manager]

1. USER VIEW (End-User)

Primary Goal: Convenience & Usability

- Platform for executing applications
- Interfaces: GUI, CLI, APIs
- Focus: Running apps, files, settings
- User concerns: Responsiveness, ease of use

Example: User Perspective

User writes code:

```
a = b + c;
```

User only cares about output correctness and fast execution!

2. SYSTEM VIEW (Resource Manager)

Primary Goal: Efficiency & Control

- OS as a master Resource Allocator
- Controls CPU, Memory, I/O devices
- Enforces Security, Stability & Isolation
- Target audience: SysAdmins, Engineers

Example: System Perspective

Inside OS & Hardware:

1. Memory allocated for a, b, c
2. Values loaded in registers (R1, R2)
3. CPU executes ADD, result written to RAM
4. Protection prevents illegal memory access

Fig 3.1: Dual Perspectives of Operating System

3.3 Numerical / Computational Example & Step-by-Step Thought Process: $a = b + c$

Image 3 handwritten note: $a = b + c$. Aaiye iska complete thought process samjhein:

Scenario: User writes `int b = 10; int c = 20; int a = b + c;`

- User View Thought Process:** User chahta hai ki `10 + 20` add hokar turant `30` mil jaye. User ko underlying hardware address se matlab nahi hai.
- Translators Role (Ref Concept 2.2):** Compiler ne iss instruction ko machine code mein toda:
 - `LOAD R1, [Addr_B]` (Value 10 in Register R1)
 - `LOAD R2, [Addr_C]` (Value 20 in Register R2)
 - `ADD R3, R1, R2` (ALU performs addition: 30)
 - `STORE [Addr_A], R3` (Value 30 stored in RAM address A)
- System View (Resource Manager) Thought Process:**
 - Process Allocation:* OS iss code ko PID deta hai aur isolated RAM allot karta hai.
 - Memory Protection:* MMU check karta hai ki process apni boundary ke bahar na jaye.
 - CPU Scheduling:* OS process ko CPU cycle allot karta hai aur context switch ke samay register state save karta hai.
 - I/O Signaling:* Final output screen driver ke zariye user ko deliver hota hai.

3.4 User View vs System View Comparison Table

Feature	User View (End-User)	System View (Resource Manager)
Main Objective	Convenience & Usability	Efficiency & Resource Optimization
Interaction Mode	GUI, CLI, Desktop Apps	System Calls, Interrupts, Drivers
Focus Area	App execution & output response	Memory Allocation, CPU Scheduling, Security
Perspective on $a=b+c$	"Sirf 30 answer aana chahiye."	"RAM allocate hui, Registers load hue, MMU verified!"

4. Kernel aur uske Core Operations (With Types of Kernel)

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concepts: Concept 1.1 mein humne dekha tha ki OS hardware manage karta hai aur Concept 3.2 (System View) mein Kernel ko core component bataya gaya tha. Yeh topic **Image 4** par aadharit hai jo Kernel ke **5 Fundamental Operations** aur **3 Types of Kernel** ko detail karta hai.

4.1 Kernel kya hota hai (The Core Bridge)

- **Heart of the OS:** Kernel OS ka core program hai jo boot hote hi RAM mein load hota hai aur system chalne tak active rehta hai.
- **Bridge Role:** Yeh hardware aur software ke beech direct bridge hai. Applications hardware ko direct touch nahi kar sakte; unhe Kernel se **System Calls** ke through permission leni padti hai.

4.2 Key Functions / Operations of Kernel (5 Pillars)

1. **Process Management:** CPU time allot karna (CPU Scheduling), process creation, coordination aur termination ensure karna.
2. **Memory Management:** RAM allocation/deallocation, Memory Protection (isolation), aur Virtual Memory handle karna.
3. **Device Management:** Hardware peripherals ke device drivers handle karna aur uniform I/O interface provide karna.
4. **File System Management:** Storage devices (SSD/HDD) par files create, read, write aur organize karna.
5. **Security & Access Control:** User permissions verify karna, unauthorized access rokna aur system integrity maintain karna.

Fig 4.1: Kernel Architecture & Core Operations

Reference: Image 4 [Kernel Functions & 3 Types Comparison]

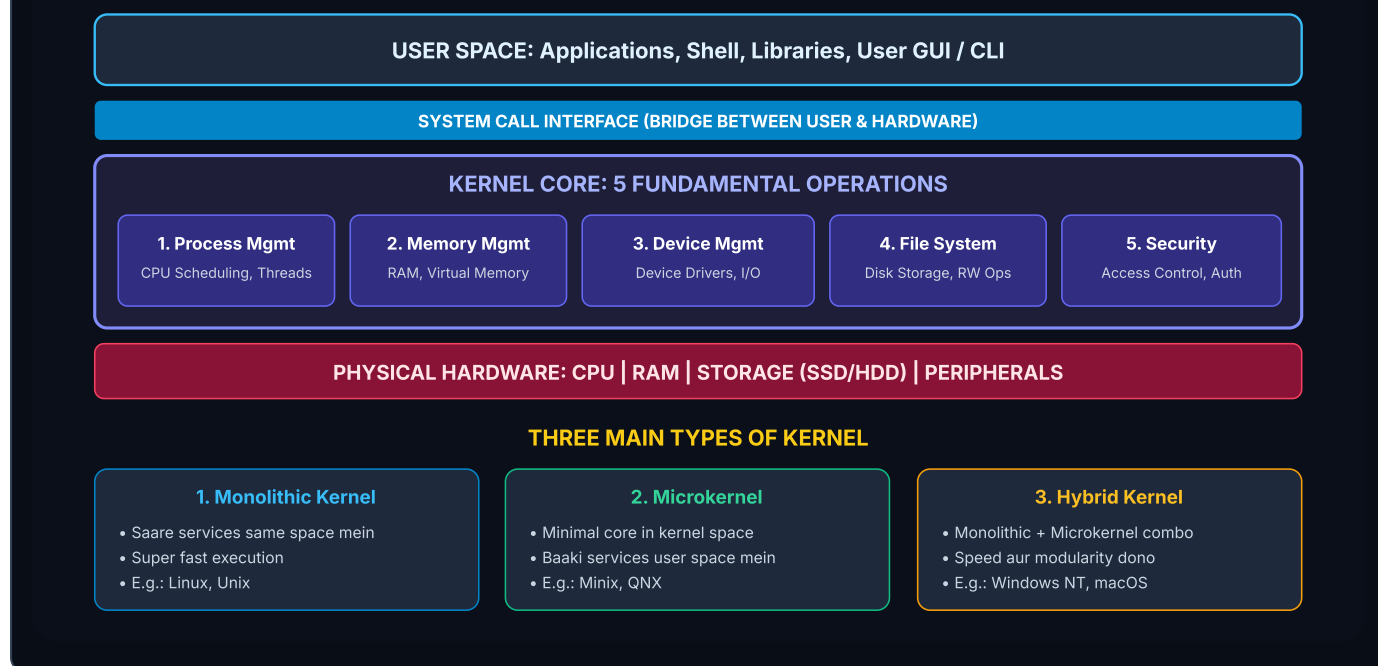


Fig 4.1: Kernel Architecture, 5 Operations & 3 Kernel Types

4.3 Types of Kernel (Image 4 Reference)

4.3.1 Monolithic Kernel

- Definition:** Saare OS services (Process, Memory, File System, Drivers) **ek hi address space (Kernel Space)** mein run karte hain.
- Pros:** Super fast execution (direct memory function calls).
- Cons:** Agar ek driver crash ho, toh poora OS crash ho sakta hai.
- Examples:** Linux, Traditional Unix.

4.3.2 Microkernel

- Definition:** Kernel space ko **bahut chhota (minimal)** rakhta hai. Baaki services User Space mein servers ke roop mein chalti hain.
- Pros:** Highly secure aur crash-resistant (Driver crash hone se OS nahi marta).
- Cons:** IPC (message passing) overhead ki wajah se thoda slow.
- Examples:** Minix, QNX, L4.

4.3.3 Hybrid Kernel

- Definition:** Monolithic ki high speed aur Microkernel ki modularity ko combine karta hai.
- Examples:** Windows NT (Windows 10/11), macOS (XNU).

4.4 Kernel Types Comparison Table

Paimana	Monolithic Kernel	Microkernel	Hybrid Kernel
---------	-------------------	-------------	---------------

Address Space	Saare services Kernel Space mein	Sirf core kernel space mein, baaki User Space	Mixture (Critical parts in kernel space)
Speed	Bahut Fast	Thoda Slow (IPC overhead)	Fast & Balanced
Fault Tolerance	Low (Driver crash = OS crash)	Very High (Stable)	High
Real Examples	Linux, Unix	Minix, QNX	Windows NT, macOS

5. Computer System Organization & Architecture

Computer Science Fundamentals | Module 2: System Organization | CS_Fundamental-00

Reference to Previous Concepts: Concept 2.1 & 2.3 mein humne Hardware resources (CPU, Memory, I/O) dekhe the. Yeh topic **Image 5** par aadharit hai jisme modern system ka **Common Bus Architecture, Device Controllers**, aur bottom slide mein **CPU Internal Architecture (ALU, CU, Registers, I/O Modules)** explain kiye gaye hain.

5.1 Modern Computer System Operation & Common Bus

- **Structure:** Modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a **Common System Bus** that provides access to shared memory.
- **Multicore Architecture (Image 5 Note):** Single chip par multiple execution cores hone se parallel computing aur high throughput milta hai.

5.2 Device Controllers & Concurrent Execution

- **Specialized Controllers:** Disk Controller (Storage), USB Controller (Peripherals), Graphics Adaptor (Video output).
- **Memory Cycle Competition:** CPU aur Device Controllers **concurrently (ek sath)** execute hote hain aur shared memory access ke liye compete karte hain.

5.3 Memory Controller & Synchronization

Orderly access ensure karne ke liye **Memory Controller** provide kiya jata hai jo memory access ko synchronize karta hai aur data clash rokta hai.

Fig 5.1: Computer System Organization & CPU Architecture

Reference: Image 5 [Modern System Bus & Internal CPU Operations]

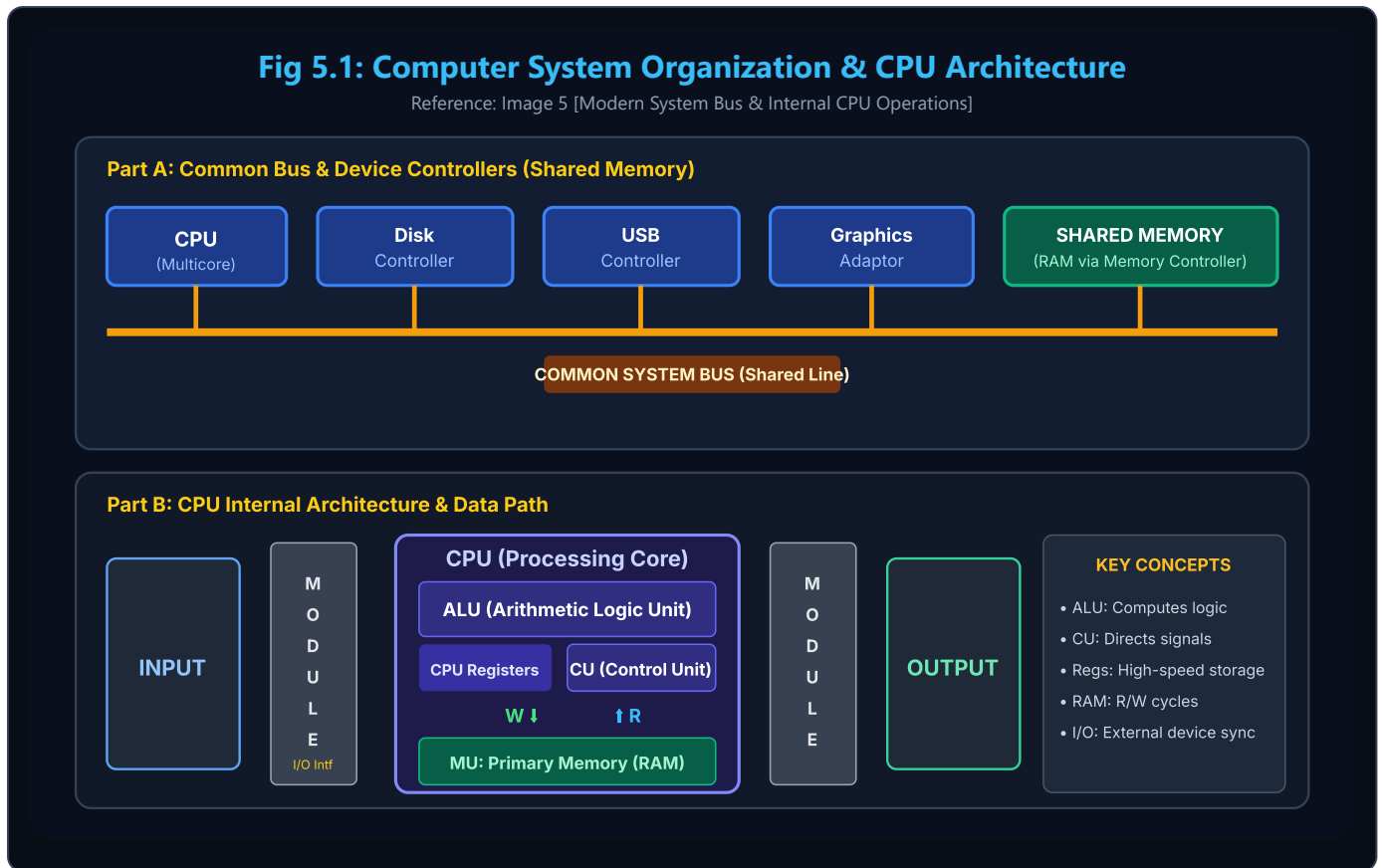


Fig 5.1: Common Bus Architecture & CPU Internal Data Path

5.4 CPU Internal Architecture (Image 5 Bottom Handwritten Diagram)

- **ALU (Arithmetic Logic Unit):** Arithmetic operations (+, -, *, /) aur logical decisions (AND, OR, NOT).
- **CU (Control Unit):** Data timing aur instruction sequencing control karta hai.
- **CPU Registers:** High-speed internal storage for immediate data.
- **Memory Unit (MU / RAM):** Read (R) aur Write (W) operations.
- **I/O Interface Modules:** Input data receive karna aur processed output display/devices tak bhejna.

6. Bootstrap Program aur Booting Process

Computer Science Fundamentals | Module 2: System Booting | CS_Fundamental-00

Reference to Previous Concepts: Concept 4.1 mein humne dekha tha ki Kernel OS ka core engine hai jo boot par RAM mein load hota hai. Yeh topic **Image 6 & Image 7** par aadharit hai jo batata hai ki RAM khaali hone par Kernel ko kaun aur kaise load karta hai!

6.1 Bootstrap Program (Bootstrap Loader) kya hai?

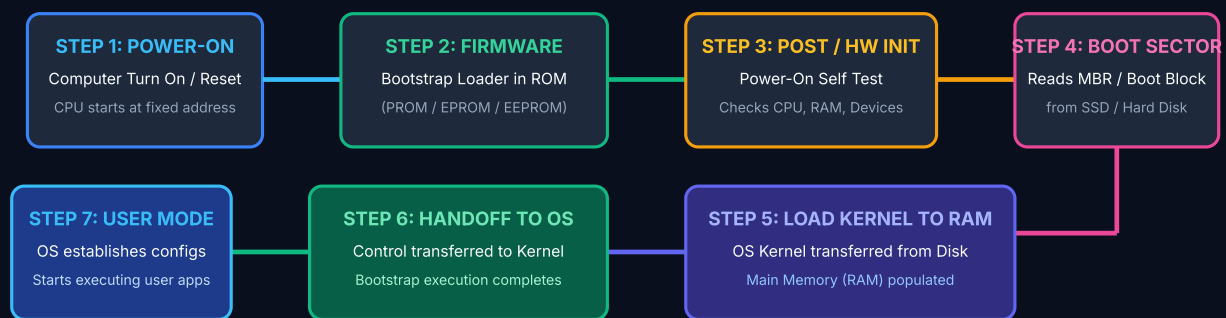
- **Initial Startup Code:** Chhota sa code jo non-volatile **Firmware / ROM** mein store hota hai.
- **Primary Purpose:** System hardware initialize karna aur storage drive se OS Kernel ko Main Memory (RAM) mein load karke CPU ka control usse sonpna.

6.2 Firmware & ROM Types (Image 6 Handwritten Note)

- **ROM:** Permanent Read-Only Memory.
- **PROM / EPROM:** Programmable aur UV-erasable ROM chips.
- **EEPROM / Flash:** Electrically Erasable Programmable ROM (Modern BIOS / UEFI chips).

Fig 6.1: Bootstrap Process & System Booting Sequence

Reference: Image 6 & 7 [Firmware, POST, Boot Block, Kernel Handoff]



Summary of Firmware Terminology (Image 6 Note):

ROM (Read Only Memory): Permanent non-volatile storage jisme initial bootstrap code rehta hai. **PROM / EPROM / EEPROM:** Programmable / Erasable ROM chips jisme firmware updates flash kiye jate hain (BIOS / UEFI). **Boot Sector / Boot Block:** Storage device ka first physical sector (MBR/GPT) jo OS bootloader code rakhta hai. **Kernel Handoff:** Ek baar Kernel RAM mein load hua, firmware CPU ka control Kernel ko de deta hai aur kaam khatam kar deta hai.

6.3 Step-by-Step Working of Bootstrap Program (Images 6 & 7)

1. **Power-On or Reset:** CPU fixed startup routine se instruction fetch karna shuru karta hai.
2. **POST (Hardware Diagnostics):** CPU registers, memory aur peripheral devices ki integrity check hoti hai.
3. **Loading the OS:** Boot device (SSD/HDD) ke **Boot Sector / Boot Block** ko read karke Kernel ko RAM mein transfer karta hai.
4. **Handoff to Kernel:** Kernel RAM mein aate hi CPU execution control Kernel ko handoff kar diya jata hai.
5. **User Mode Transition:** OS system configurations establish karta hai aur user applications chalane ke liye **User Mode** mein enter kar jata hai.

7. Dual-Mode Operation (User Mode vs Kernel Mode)

Computer Science Fundamentals | Module 2: OS Protection | CS_Fundamental-00

Reference to Previous Concepts: Concept 3.1 & 3.2 mein User View vs System View dekha tha, aur Concept 4.1 mein Kernel ko bridge dekha tha. Yeh topic **Image 7 (bottom) & Image 8 (top)** par aadharit hai jo **User Mode vs Kernel Mode** aur code `printf("Hello");` ka transition explain karta hai.

7.1 Dual-Mode Operation ki Zaroorat (Protection & Safety)

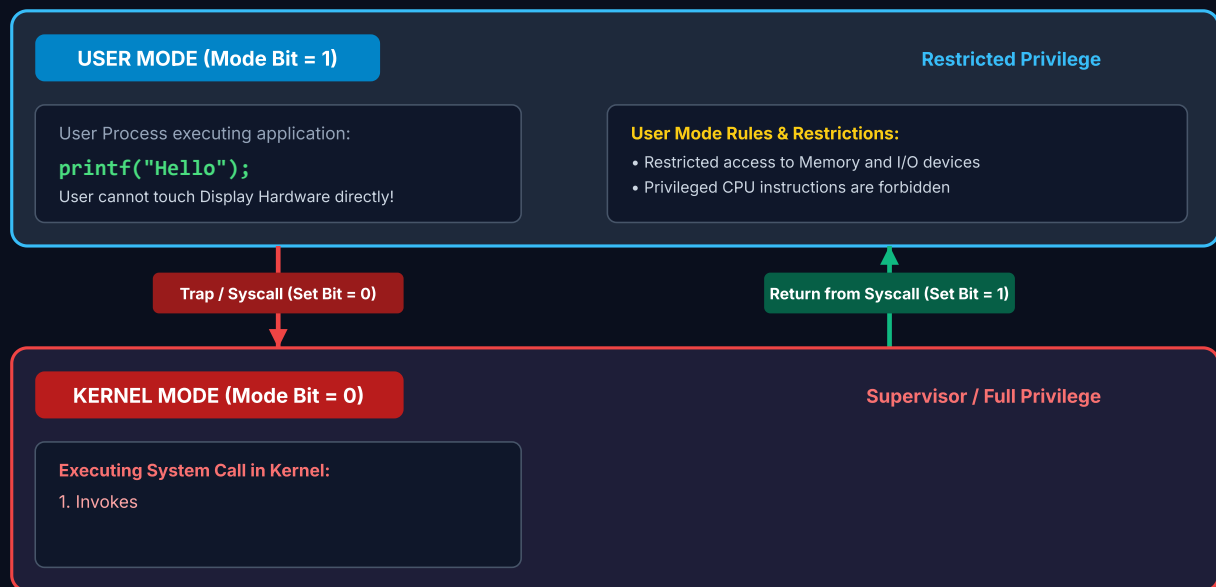
Agar koi normal software hardware ya RAM ko directly modify kar sake, toh ek glitch poore computer ko crash kar sakta hai. Isliye hardware do distinct privilege modes provide karta hai.

7.2 User Mode vs Kernel Mode

- **User Mode (Restricted Privilege - Mode Bit = 1):** User processes execute hote hain. Hardware resources direct access nahi kiye ja sakte.
- **Kernel Mode (Supervisor / Full Privilege - Mode Bit = 0):** OS Kernel execute hota hai. Hardware, RAM, registers aur privileged instructions ka full access hota hai.

Fig 7.1: Dual-Mode Operation & System Call Transition

Reference: Image 7 (Bottom) & Image 8 (Top) [User Mode → Kernel Mode via printf]



`sys_write()` 2. Sends data to Video Buffer / Display 3. Writes "Hello" to Terminal Screen Kernel Mode Privileges: • Full control over hardware resources & memory • Direct execution of privileged machine instructions • Manages hardware interrupts, I/O ports & MMU

Fig 7.1: Dual Mode Transition & System Call Trace

7.3 Handwritten Example & Step-by-Step Thought Process: printf("Hello");

Image 7 bottom note: USER MODE -----> Kernel mode | printf("Hello");

Thought Process Trace:

1. User Mode Execution (Mode Bit = 1):

User program `printf("Hello")` call karta hai. Library pehchanti hai ki display screen par write karna ek hardware operation hai jo User Mode mein allowed nahi hai.

2. Trap & Mode Switch (1 → 0):

Program `sys_write()` system call execute karta hai. Hardware Trap trigger hota hai aur CPU Mode Bit ko `1` se `0` (Kernel Mode) switch kar deta hai.

3. Kernel Mode Execution (Mode Bit = 0):

Kernel parameters verify karta hai aur display controller ke video buffer mein safely write karta hai. Screen par "Hello" print ho jata hai.

4. Return to User Mode (0 → 1):

Kernel return instruction deta hai. Hardware Mode Bit ko wapas `1` (User Mode) set kar deta hai aur user app aage chalta hai.

7.4 User Mode vs Kernel Mode Comparison Table

Paimana	User Mode	Kernel Mode
Mode Bit	1	0
Privilege	Restricted Access	Full Hardware & Memory Access
Executing Code	User Applications (Chrome, Word, Python)	Operating System Kernel
Hardware I/O	Direct I/O Forbidden (Needs System Call)	Direct I/O Permitted

8. Functions of Operating System: Memory Management

Computer Science Fundamentals | Module 2: Memory Management | CS_Fundamental-00

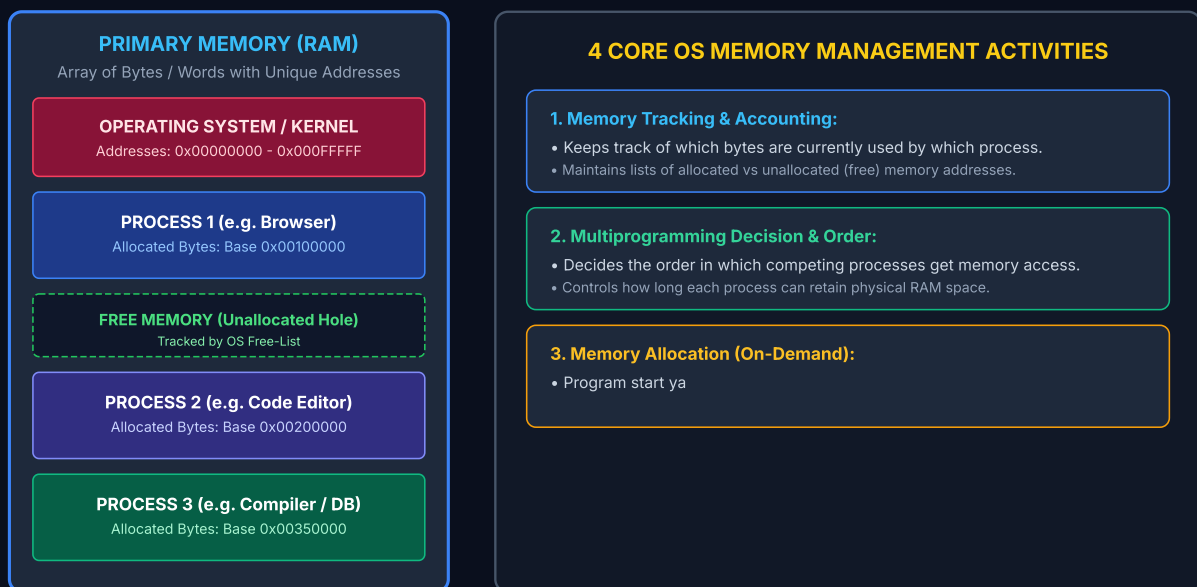
Reference to Previous Concepts: Concept 4.2.2 mein humne Kernel ke 5 core functions mein Memory Management dekha tha, aur Concept 3.3 ke numerical example ($a = b + c$) mein RAM allocation samjha tha. Yeh topic **Image 8 (bottom slide)** par aadharit hai.

8.1 Primary Memory / Main Memory (RAM) ki Sanrachna

- **Array of Bytes / Words:** Main memory ek bahut bada array of bytes/words hoti hai jisme har byte/word ka ek unique numerical memory address hota hai.
- **Direct CPU Access:** Fast storage jise CPU system bus ke zariye direct read/write access karta hai.
- **Execution Shart:** Kisi bhi program ko execute hone ke liye uska pehle **Main Memory (RAM)** mein load hona anivarya (mandatory) hai.

Fig 8.1: Main Memory Array Architecture & OS Management Activities

Reference: Image 8 [Primary Memory Array of Bytes, Multiprogramming & Allocation]



`malloc()` call par RAM bytes allot karta hai. • Sets Base and Limit registers in MMU for safety. 4. **Memory Deallocation & Reclamation:** • Process exit hone par ya I/O wait ke dauran memory free karta hai. • Reclaims blocks into free-list to prevent Memory Leaks.

Fig 8.1: Primary Memory Array & 4 Core OS Activities

8.2 OS ki 4 Core Memory Management Activities (Image 8 Reference)

1. **Memory Tracking & Accounting:** Track karta hai ki kaun se bytes kis process dwara use ho rahe hain, kaun se allocated hain aur kaun se free hain.

2. **Multiprogramming Access Decisions:** Multiprogramming environment mein decide karta hai ki competing processes ko kis order mein aur kitni der tak memory access milega.
3. **Dynamic Memory Allocation:** Process request par RAM bytes allot karta hai aur MMU base/limit registers set karta hai.
4. **Memory Deallocation & Reclamation:** Process terminate hone par memory reclaim karta hai taaki Memory Leaks na hon.

8.3 Numerical / Computational Example & Step-by-Step Thought Process: Memory Addressing & Allocation

Scenario: Total RAM = 1024 KB. OS Reserved = 0-199 KB (200 KB). Free = 200-1023 KB (824 KB).

1. **Process P1 Request (300 KB):** OS P1 ko 200 se 499 KB allot karta hai. Base Register = 200 KB, Limit Register = 300 KB. Free memory bachi = 524 KB (500-1023 KB).
2. **Process P2 Request (250 KB):** OS P2 ko 500 se 749 KB allot karta hai. Base Register = 500 KB, Limit Register = 250 KB. Free memory bachi = 274 KB.
3. **Hardware Protection Verification:** Agar P2 address 450 KB access karne ki koshish kare, toh MMU hardware check karega: $450 < \text{Base}(500)$ → **Illegal Access!** Trap trigger karke process kill ho jayega (Memory Protection!).
4. **Deallocation:** P1 terminate hone par 300 KB block free-list mein return hota hai aur agle processes ke liye ready ho jata hai.

9. Memory Hierarchy & Caching Deep Dive

Computer Science Fundamentals | Module 3: Memory Subsystem | CS_Fundamental-00

Interview Memory Hook: "Jo memory jitni fast hogi, woh utni hi mehngi aur chhoti hogi. Jo memory sasti aur badi hogi, woh dheemi hogi." Is trade-off ko solve karne ke liye **Memory Hierarchy** aur **Multi-Level Cache** bani!

9.1 Memory Hierarchy Pyramid & Core Trade-offs

- **Level 1 (Top) - CPU Registers:** Fastest (0.25-0.5 ns), small (few KB), expensive.
- **Level 2 - Cache Memory (SRAM):** L1, L2, L3 caches (1-20 ns).
- **Level 3 - Main Memory (DRAM):** Primary RAM (50-100 ns, 8-64 GB), volatile.
- **Level 4 - Secondary Storage:** NVMe / SSD / HDD (Microseconds to milliseconds, Terabytes).
- **Level 5 (Bottom) - Magnetic Disks & Cold Storage:** Backup / Tapes.

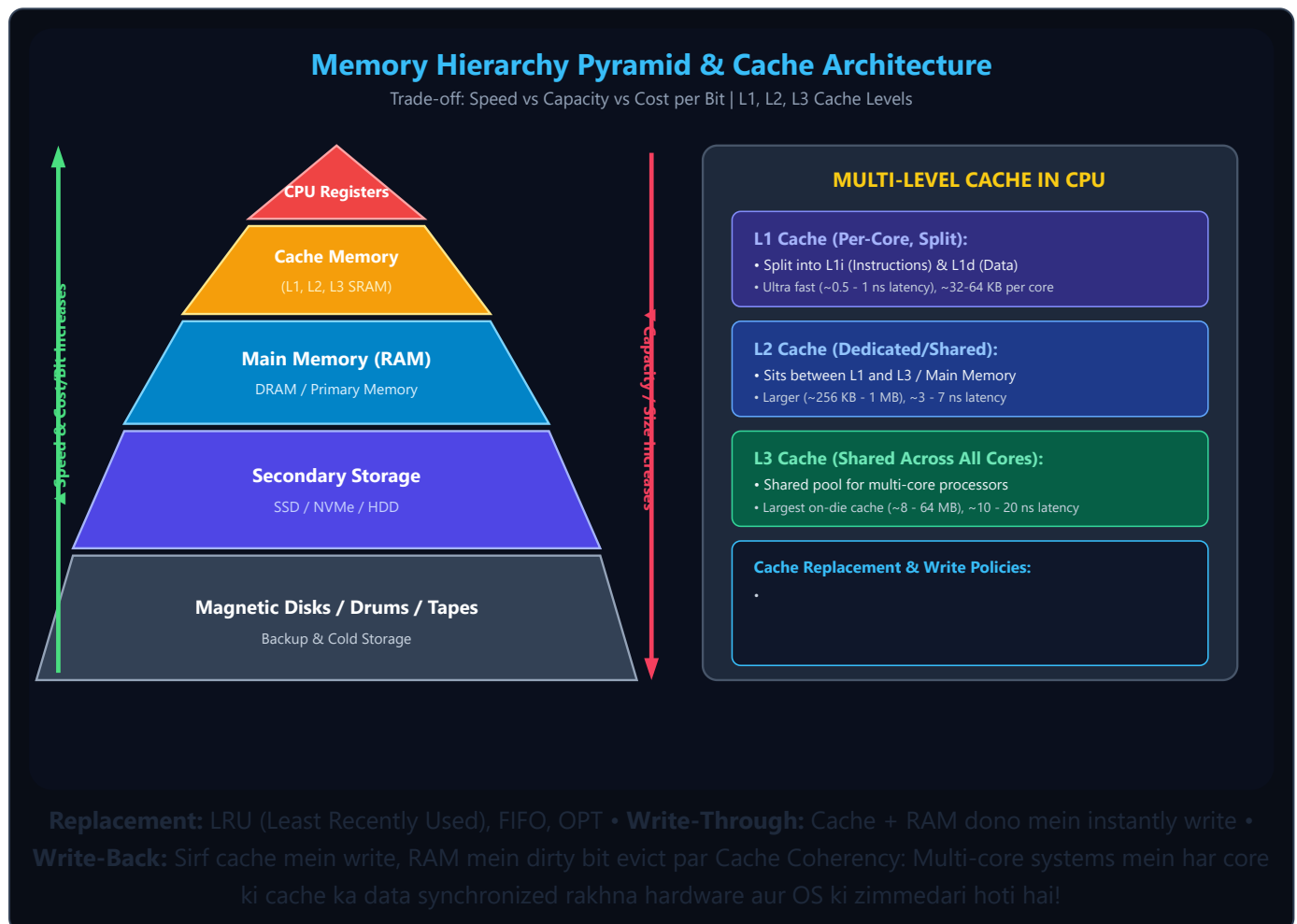


Fig 9.1: Memory Hierarchy Pyramid & Multi-Level Cache Architecture

9.2 Caching & Principle of Locality

- **Temporal Locality:** Jo data abhi access hua hai, woh jald hi dobara access hoga (Loops, counters).

- **Spatial Locality:** Jo memory address access hua hai, uske agal-bagal ke addresses bhi jald access honge (Arrays, sequential code).
- **Average Memory Access Time (AMAT):** $AMAT = T_{cache} + (1 - H) * T_{RAM}$

9.3 Multi-Level Caches (L1, L2, L3)

- **L1 Cache:** Per-core dedicated, split into L1i (Instructions) & L1d (Data). ~32-64 KB, ~1 ns.
- **L2 Cache:** Intermediate buffer between L1 and RAM. ~512 KB - 2 MB per core.
- **L3 Cache:** Shared across all CPU cores on die. ~8 - 64 MB, ~10-20 ns.

9.4 Cache Policies: Replacement & Writes

- **Replacement:** LRU (Least Recently Used), FIFO, Random, OPT.
- **Write-Through:** Cache aur RAM dono mein instantly write (consistent, slower).
- **Write-Back:** Sirf cache mein write, Dirty Bit set. RAM mein evict hone par likhta hai (fastest).
- **Cache Coherency:** Multi-core mein sabhi cores ke cache data ko synchronize rakhna (MESI protocol).

10. Processor Management & CPU Scheduling

Computer Science Fundamentals | Module 3: Process Subsystem | CS_Fundamental-00

Interview Memory Hook: Program hard disk par rakhi ek band kitaab hai. Jab use khol kar execute karte hain, tab woh **Process** banti hai! OS ka Processor Manager decide karta hai ki kaun sa process kab tak CPU core par chalega.

10.1 Program vs Process & PCB

- **Program:** Passive code file on disk (app.exe).
- **Process:** Active entity in execution with Program Counter, Stack, Heap, Registers, and PCB.
- **PCB (Process Control Block):** Process ID, State, CPU Registers, Memory limits.

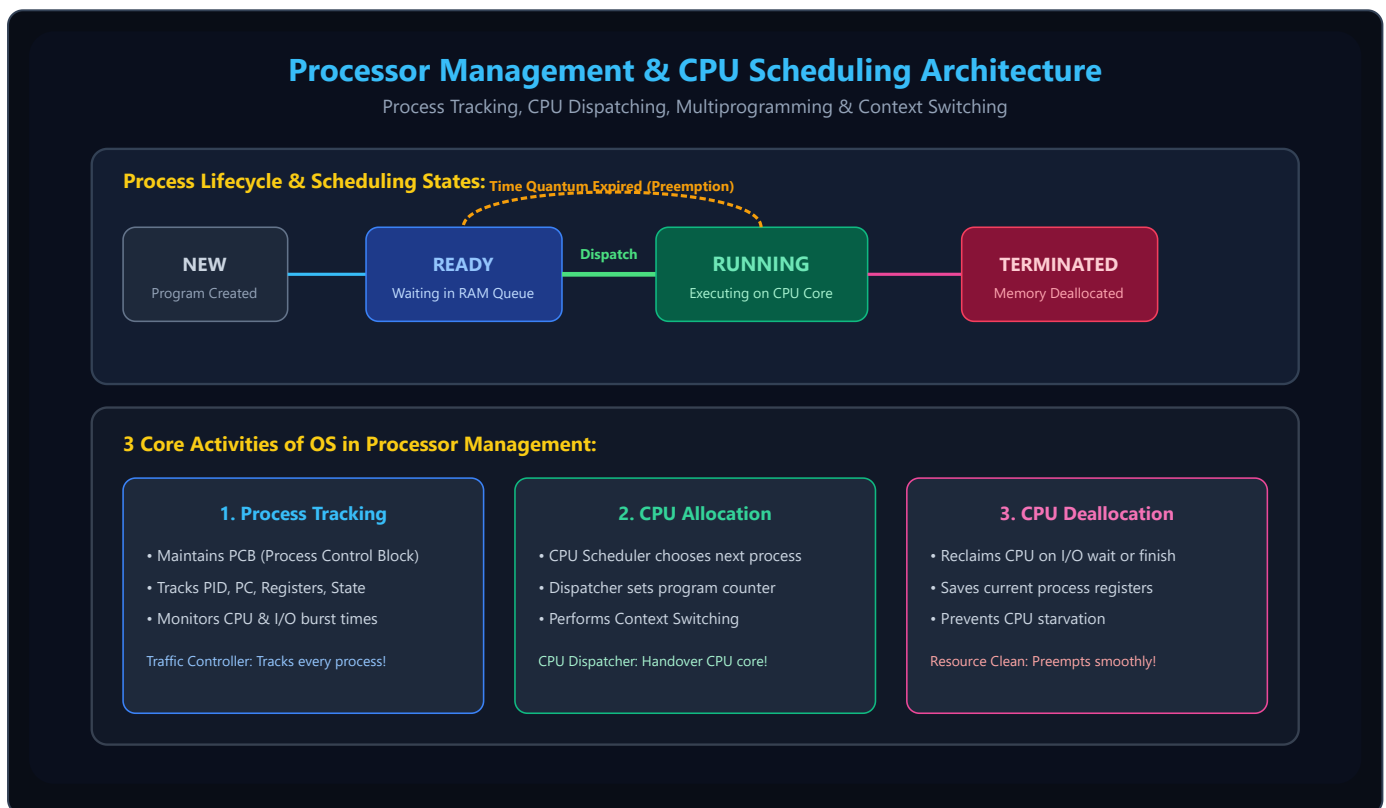


Fig 10.1: Process Lifecycle & CPU Scheduling Dispatcher

10.2 Multiprogramming & Scheduling

CPU utilization maximize karne ke liye jab chal raha process I/O wait karta hai, OS turant doosre ready process ko CPU dispatch kar deta hai.

10.3 OS ki 3 Core Activities

1. **Process Tracking:** PCB aur process states (New, Ready, Running, Waiting, Terminated) monitor karna.
2. **CPU Allocation:** Ready queue se process chun kar CPU core assign karna (Context Switch).
3. **CPU Deallocation:** Task finish hone par ya time quantum expire hone par CPU reclaim karna.

10.4 Numerical / Interview Thought Process: Context Switch Latency Trace

Context Switch Execution Steps (Process P1 → P2):

1. Timer interrupt trigger hota hai ($t = 0 \mu s$), CPU enters Kernel Mode.
2. Current process P1 ke registers aur program counter PCB1 mein save hote hain ($t = 1 \mu s$).
3. Scheduler agla process P2 choose karta hai ($t = 2 \mu s$).
4. PCB2 se saved state CPU registers mein reload hoti hai ($t = 3 \mu s$).
5. Mode bit 1 set karke P2 execute hota hai ($t = 4 \mu s$). Total overhead = $4 \mu s$ non-productive time!

11. Device Management & File System Management

Computer Science Fundamentals | Module 3: Storage & I/O Subsystem | CS_Fundamental-00

Interview Memory Hook: Hazaaron alag-alag companies ke hardware ke liye developer ko alag code na likhna pade, isliye OS **Device Drivers** deta hai. Aur storage ke raw blocks ko sundar folders mein **File System** badalta hai!

11.1 Device Management (I/O Subsystem)

- **Device Drivers:** Hardware-specific translators jo generic commands (read/write) ko hardware signals mein badalte hain.
- **Uniform Interface:** Application code ko hardware details se decouple karta hai.
- **Activities:** Tracking connected devices, designating I/O controllers, scheduling device access, safe allocation & deallocation.

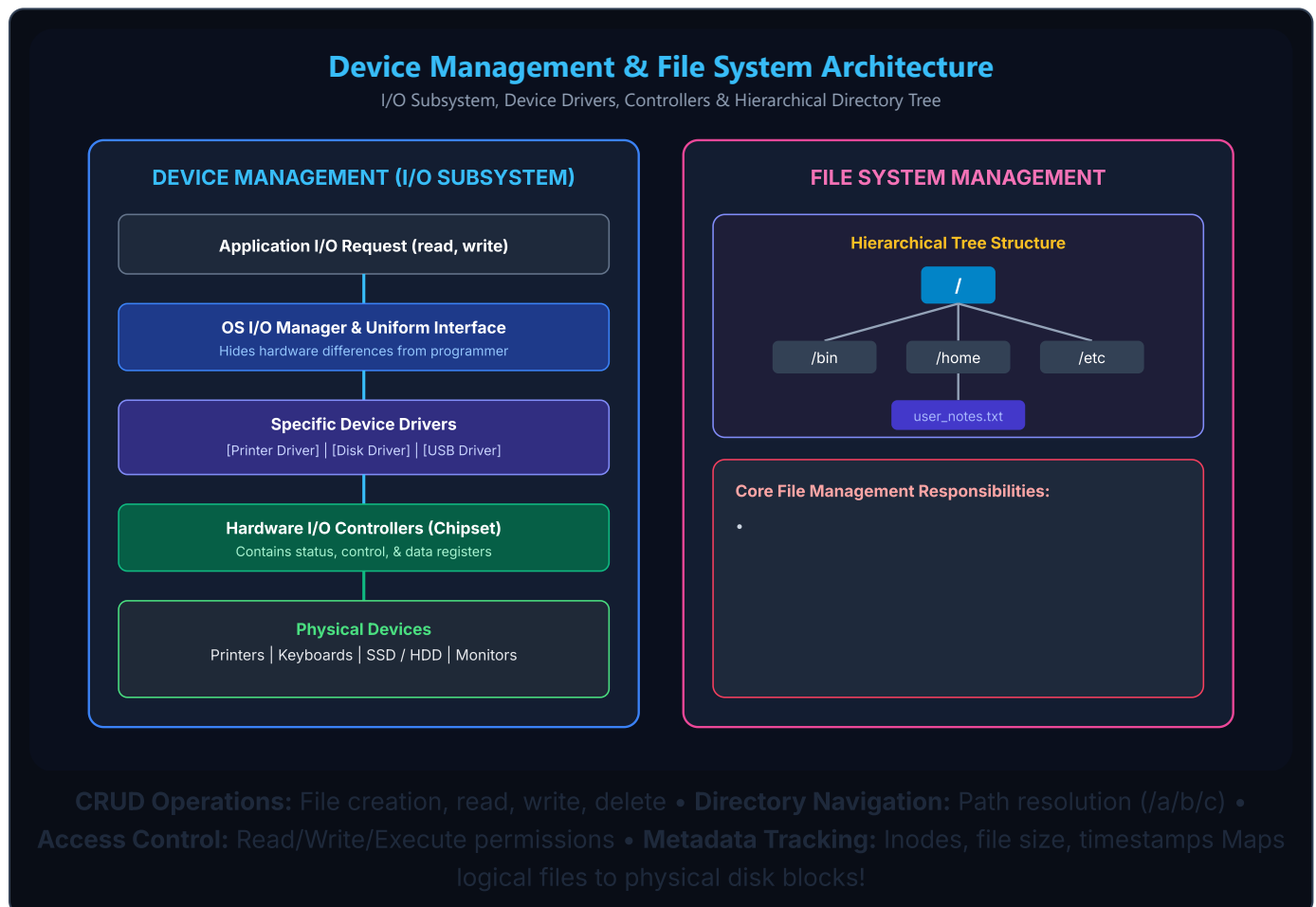


Fig 11.1: Device Driver Subsystem & Hierarchical File System Tree

11.2 File System Management

- **Logical Abstraction:** Physical disk blocks ko logical Files & Directories mein map karna.

- **Hierarchical Tree:** Root directory /, subdirectories, path resolution.
- **File Operations:** Create, Read, Write/Update, Delete, Open, Close.
- **Metadata & Inodes:** File size, permissions (rwx), block pointers, timestamps.

11.3 Step-by-Step Thought Process: File Read Journey (open("notes.txt", "r"))

Execution Trace:

1. **System Call:** User mode app invokes `sys_open()`.
2. **Path Resolution:** Directory traversal: `/` → `home` → `shivam` → `notes.txt`.
3. **Permission & Inode Check:** OS verifies read permission bit.
4. **Block Translation:** Inode maps logical offset to physical disk blocks.
5. **Device Driver I/O:** Disk controller reads blocks into RAM buffer.
6. **File Descriptor Return:** OS returns fd index to user application.